



Mash-Up
iOS Team

2026.04.14

Claude code 입문편

Mash-Up iOS Team Seminar





목차

iOS Seminar

1	Claude.md	프로젝트 컨텍스트의 뿌리
2	Plan Mode	계획 → 검토 → 실행
3	유용한 명령어	/resume, /output-style, /export
4	컨텍스트 관리	Lazy Loading · 한 세션 한 피쳐
5	워크플로우	WAT 프레임워크



목차

iOS Seminar

- | | | |
|----|-----------|------------------------|
| 6 | Skills | AI에게 주는 업무 매뉴얼 |
| 7 | Sub-Agent | 병렬 실행 & 컨텍스트 격리 |
| 8 | Hooks | 이벤트 기반 자동화 엔진 |
| 9 | 고급 팁 | 멀티 인스턴스, Worktree, MCP |
| 10 | 통합 데모 | 8단계 파이프라인 워크플로우 |



Claude.md 넣어야 할 내용



절대 규칙

위반하면 안 되는 금지 사항
반드시 맨 위에 배치



아키텍처

폴더 구조, 기술 스택



도메인

비즈니스 용어, 데이터 흐름



빌드/테스트

개발 서버 / 테스트 / 배포 명령어



컨벤션

네이밍 · 커밋 · 패턴 규칙



300줄 이하로 유지하세요! 너무 길면 매 대화마다 토큰을 낭비합니다



Claude.md 계층 구조

글로벌 설정

`~/.claude/claude.md`

모든 프로젝트에 공통으로 적용되는 규칙
예) "항상 한국어로 답변해줘"

프로젝트 설정

`프로젝트루트/claude.md`

해당 프로젝트에만 적용
아키텍처 · 컨벤션 · 기술 스택 등 프로젝트 고유 정보

 글로벌엔 공통 규칙을, 프로젝트엔 해당 프로젝트의 아키텍처와 컨벤션을 분리해서 관리하세요



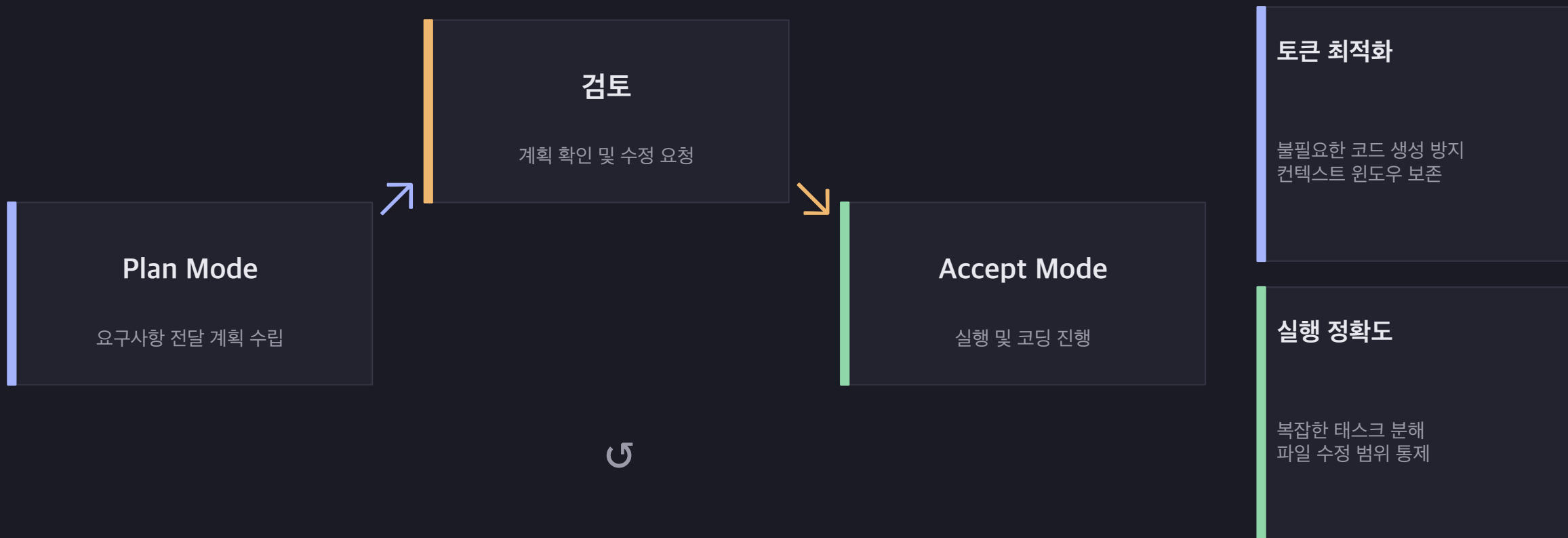
Claude.md 계층 구조 — 트리





Plan Mode ↔ Accept Mode

Shift + Tab으로 두 모드를 전환 · 새로운 작업은 항상 Plan Mode로 시작





유용한 명령어

<code>/resume</code>	이전 세션 복구
<code>/output-style</code>	응답 스타일 설정 (학습 모드, 간결 모드 등)
<code>/export</code>	채팅 내보내기 (Cursor 등으로 옮길 때 유용)
<code>! + 명령어</code>	bash 명령 바로 실행
<code>/clear</code>	현재 컨텍스트 초기화 (새 세션 시작)

```
! git push
```

```
! for bash mode
```

! Skills로 통합?

커스텀 슬래시 명령어

자주 쓰는 작업을 명령어로 등록

```
.claude/
└─ commands/
    └─ review_script.md
        └─ → /review_script
    └─ deploy_staging.md
        └─ → /deploy_staging
```

파일명이 그대로 명령어 이름이 됨



Status line

범주	표시 가능
모델 정보	이름, ID, 버전
디렉토리·프로젝트	CWD, project dir, 추가 dirs
Git	브랜치, worktree 이름, staged/modified/untracked 수, ahead/behind
컨텍스트	사용률 %, 남은 %, 토큰 수, 캐시 토큰
비용·시간	세션 비용, 경과 시간, API 시간, 코드 줄 변경수
Rate Limit	5시간·7일 사용률, 리셋 시각
Worktree	이름, 경로, 브랜치, 원본 브랜치
Agent·Vim	에이전트 이름, vim 모드
외부 정보	셸 스크립트니까 뭐든지 — 날씨·시간·주가·Jira 티켓·SwiftLint 에러 수 등



Core:main UI:main | [██████████] 4% | Opus 4.6 (1M context)
▶▶ accept edits on (shift+tab to cycle)



main | ctx [██████████] 5% | Opus 4.6 (1M context)
5h [██████████] 50% (42m) | 7d [██████████] 8% (2d 23h)



🦥 컨텍스트 관리 — Lazy Loading

참조만 두고, 상세 내용은 별도 파일로 분리

❌ Bad

하나의 거대한 Claude.md

```
Claude.md
├─ 절대 규칙
├─ 아키텍처 (전체)
├─ UI 모듈 상세
├─ Core 모듈 상세
├─ Network 상세
└─ ... (1500줄)
```

✅ Good

폴더별 Claude.md 분리

```
Claude.md (루트, 짧게)
├─ UI/
│   └─ Claude.md
├─ Core/
│   └─ Claude.md
└─ Network/
    └─ Claude.md
```

→ Claude가 해당 폴더 작업 시에만 그 폴더의 Claude.md를 읽어 컨텍스트 bloat 방지



컨텍스트 윈도우 관리

핵심 원칙: 한 세션 = 한 피쳐

- 하나의 세션에서 여러 기능 연달아 구현하지 않기
- 깨끗한 컨텍스트가 부풀어진 컨텍스트보다 항상 낫다

실전 워크플로우

- 1 Plan Mode에서 전체 작업을 설계하고 단계를 나눈다
- 2 /clear 또는 새 세션을 연다
- 3 플랜의 첫 번째 단계만 구현한다
- 4 완료되면 다시 /clear → 다음 단계로

반복 사이클





△ 컨텍스트 관리 — Mermaid 아키텍처 정리

별도 파일에 정리하고 Claude.md에서 참조

```
docs/architecture.mermaid
```

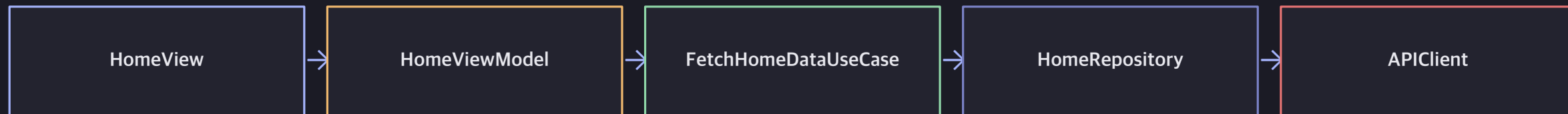
```
graph LR
```

```
A[HomeView] --> B[HomeViewModel]
```

```
B --> C[FetchHomeDataUseCase]
```

```
C --> D[HomeRepository]
```

```
D --> E[APIClient]
```



Claude.md에는 "아키텍처는 docs/architecture.mermaid 참조"라고만 적어두자



🪶 컨텍스트 관리 — 무거운 작업 오프로드

📜 스크립트로 오프로드

무거운 데이터 처리를 대화 안에서 시키면 컨텍스트가 오염됨

```
# 1. 스크립트 작성하게 시키고
claude "이 csv를 정리하는
       파이썬 스크립트 만들어줘"

# 2. 결과 요약만 받기
python clean.py | head
```

Claude는 결과만 보고, 원본 데이터는 컨텍스트에 안 들어옴

🔌 사용 안 하는 MCP 비활성화

연결만 해놔도 도구 설명만으로 토큰을 크게 소모

❌ 안 쓰는데 켜둠
Figma, Notion, Linear, Slack...
→ 매 턴 수천 토큰 낭비

✅ 자주 쓰는 것만 켜기
또는 커스텀 MCP로 래핑해서 필요한 도구만 노출

💡 MCP Tool Search 사용
MCP 도구 설명이 컨텍스트 윈도우의 10%를 넘으면
자동으로 deferred loading 활성화
% 조정 가능, 항상 true 가능



Claude Code를 잘 쓰기 위한 6가지 습관



큰 변경은 반드시 Plan Mode로 시작

한 번에 다 하지 말고 단계로 쪼개기



TDD 기반 코딩

작은 변경 → 테스트 → 린트 → 커밋 → 반복



Thinking 로그 읽기

Claude가 왜 그런 결정을 했는지 추적



다른 AI에게 비평 받기

같은 결과를 다른 모델에 검토 의뢰



에러 로그를 그대로 붙여넣기

요약하지 말고 raw 로그를 통째로



로컬 TODO.md 활용

복잡한 작업의 진행 상태 추적



WAT 프레임워크

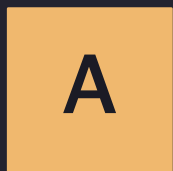
Nate Herk가 제안한 복잡한 프로젝트 관리 방법론



Workflows

작업 흐름 정의

자연어로 단계를 명확히 정의



Agents

AI 에이전트 활용

Self-healing + Sub-agent 병렬 처리



Tools

도구 조합

작고 원자적인 도구들을 조합



WAT — Workflows

코드를 쓰기 전에 작업 흐름을 글로 먼저 정의

1	API 명세 확인 + 모델 정의	DTO, Endpoint 스펙 확인
2	Data Layer 구현	DTO, Network, Repository 구성
3	Domain 구성	UseCase, Model 변환
4	Presentation 연결	ViewModel, View 바인딩
5	통합 검증	Mock + 에러/로딩 시나리오

→ 각 단계마다 테스트 작성 후 통과 확인



WAT — Agents

Self-Healing

에러가 나면 스스로 처리

- 1 로그를 읽는다
- 2 원인을 파악한다
- 3 코드를 수정한다
- 4 다시 실행한다

Sub-agent 병렬 처리

시간 절약 — 동시에 여러 작업

Sub-agent A → 테스트 작성 및 실행

Sub-agent B → 관련 문서 업데이트

Sub-agent C → 코드 린트 및 타입 체크



WAT — Tool

Tool Atomicity (도구의 원자성) — 작은 단위가 훨씬 낫다

✗ 나쁜 예

deploy-all.sh

(200줄짜리 괴물)

```
function build() { ... }  
function test() { ... }  
function deploy() { ... }  
function notify() { ... }  
function rollback() { ... }  
# 한 군데 실패하면  
# 전체가 죽음
```

✓ 좋은 예

원자적 도구들의 조합

```
scripts/build.sh      # 빌드만  
scripts/test.sh       # 테스트만  
scripts/deploy.sh     # 배포만  
scripts/notify.sh     # 알림만
```

조합·교체·디버깅 쉬움



Skills — AI에게 주는 업무 매뉴얼

반복되는 프롬프트를 SKILL.md에 정의 → 매번 같은 품질의 결과

일반 프롬프트

- 매번 입력해야 함
- 결과가 들쭉날쭉
- 팀원과 공유 어려움
- 한 번의 일회성 지시

Skills

- 한 번 만들면 자동 호출
- 품질이 항상 일관됨
- Git에 올려서 팀 공유 가능
- 재사용 가능한 업무 시스템

프롬프트가 "한 번의 지시"라면, Skills는 "재사용 가능한 시스템"



Skills 구조 & 로딩 방식

폴더 구조

```
.claude/skills/  
└─ ppt-generator/  
   └─ SKILL.md  
      (필수 - 핵심 파일)  
   └─ template.md ← 템플릿 (선택)  
   └─ examples/   ← 예제 (선택)  
   └─ scripts/    ← 스크립트 (선택)
```

2단계 로딩 방식

1단계 이름 + description만 로드 (~50-100B)
 항상 메모리에 상주하지만 가벼움

2단계 호출 시 SKILL.md 본문 + 참고 파일 전체
 실제 쓸 때만 풀 컨텍스트 로드

 평소엔 가볍게, 쓸 때만 전체 로드.
자동 컴팩션 시 본문은 정리되고 설명만 남음.



CLAUDE.md vs Skills vs MCP vs Sub-Agent

	CLAUDE.md	Skills	MCP 도구	Sub-Agent
평소 공간	전체 매번	설명만 (~50-100B)	전체 설명 매번? (MCP Tool Search)	에이전트 설명 매번
안 쓸 때 부담	항상 있음	거의 없음	항상 있음? (MCP Tool Search)	항상 있음
언제 쓰면 좋은가	핵심 규칙	반복 작업	외부 서비스 연동	대량 탐색·분석

💡 Skills는 평소엔 거의 비용이 없다가 호출될 때만 본문을 로드해서 컨텍스트가 가장 효율적



SKILL.md 작성법

```
ppt-generator/SKILL.md
```

```
name: ppt-generator
```

```
description: "PPT 발표자료 자동 생성.
```

```
  'PPT 만들어줘', '슬라이드 제작' 등을 언급할 때 반드시 사용할 것"
```

목적

주제와 청중을 입력받아 .pptx 파일을 생성한다.

절차

1. 주제·청중·시간 확인
2. 목차·슬라이드 구성 설계
3. 슬라이드별 내용 작성 → .pptx 출력

자체 검증 체크리스트

- [] 슬라이드 수가 적절한가?
- [] 핵심 메시지가 있는가?
- [] 시각 자료 지시사항이 포함되었는가?

더 “pushy”하게



description 팁



좋은 예

“PPT 발표자료 자동 생성”, “PPT 만들어줘”, “발표자료 작성”
등을 언급할 때 반드시 사용할 것



나쁜 예

“문서를 생성하는 스킬”
→ 너무 추상적

핵심 기능을 첫 문장에,
사용자가 쓸 법한 표현 3개 이상 포함



Sub-Agent — 별도 작업 공간을 가진 도우미

자기만의 컨텍스트·도구·권한을 갖고 작업하고, 메인엔 요약만 돌려줌



병렬 처리

여러 작업 동시 실행으로
전체 소요 시간 단축



컨텍스트 보호

메인 에이전트의 컨텍스트를
오염시키지 않음



전문화

각 Sub-Agent에
전문 역할 부여



재사용

한 번 만든 에이전트를
여러 워크플로우에서 활용



Sub-Agent는 다른 Sub-Agent를 생성할 수 없음 — 메인에서 체인으로 연결할 것



내장 Sub-Agent 5종

Sub-Agent	모델	권한	용도
Explore	Haiku (빠름)	읽기전용	코드 탐색·파일 검색·구조 파악
Plan	상속	읽기전용	Plan Mode 계획 수립 연구
General-purpose	상속	모든 도구	탐색+수정 필요한 복잡 다단계 작업
Bash	상속	Bash만	별도 컨텍스트에서 터미널 실행
Claude Code Guide	Haiku	읽기전용	Claude Code 기능 질문 답변

! ? 일반 tool



🔧 커스텀 에이전트 만들기

/agents 명령어 단계별 가이드

단계별 흐름

- 1 /agents 입력
- 2 Create new agent 선택
- 3 User-level / Project-level 선택
- 4 Generate with Claude → 설명 입력
- 5 도구 선택 (Read-only / 전체)
- 6 모델 선택 (Sonnet / Opus / Haiku)
- 7 저장 → 즉시 사용 가능

code-reviewer 예시

```
---  
name: code-reviewer  
description: "코드 리뷰 전문가"  
tools: Read, Grep, Glob, Bash  
model: sonnet  
---
```

호출되면:

1. git diff로 최근 변경 확인
2. 수정된 파일에 집중하여 리뷰

● 크리티컬 / ● 경고 / ● 제안



에이전트 저장 위치 & 범위

저장 위치	사용 범위	쓰는 상황
~/ .claude/agents/	내 모든 프로젝트	나만 쓰는 개인용
.claude/agents/	이 프로젝트에서만	팀과 공유할 프로젝트 전용
플러그인의 agents/	플러그인 활성화된 곳	플러그인 제공 에이전트
--agents CLI 플래그	지금 이 세션에서만	자동화 스크립트·CI 일회용

Sub-Agent vs 메인 대화 — 언제 분리할까?

Sub-Agent로 빼기

작업이 독립적 · 대량 출력 격리 필요 · 결과만 받으면 됨

메인에서 직접

왔다 갔다 필요 · 가벼운 작업 · 중간 의사결정이 잦음



⚡ Sub-Agent 실전 패턴 4가지



대량 출력 격리

테스트·로그 분석처럼 출력이 많은 작업을 위임하고 메인엔 요약만 받기



병렬 연구

독립적인 모듈을 각각 별도 Sub-Agent로 동시에 분석



에이전트 체인

code-reviewer → optimizer 순서로
순차 연결해서 파이프라인 구성



에이전트 재개

완료 후에도 이전 컨텍스트를 유지한 채
이어서 작업 가능



⚡ Hooks — Claude Code의 자동화 엔진

이벤트가 발생하면 matcher가 조건을 검사하고, 맞으면 액션이 자동 실행



방법 1 — Claude에게 요청 ★

"알림 Hook 만들어줘"라고 말하면
settings.json에 자동 추가됨

방법 2 — settings.json 직접 편집

~/.claude/settings.json (개인용)
.claude/settings.json (프로젝트용)



Hook 이벤트 타입 & JSON 구조

settings.json 예시 - 알림 Hook

```
{
  "hooks": {
    "Notification": [
      {
        "matcher": "*",
        "hooks": [
          {
            "type": "command",
            "command": "terminal-notifier ..."
          }
        ]
      }
    ]
  }
}
```

4가지 이벤트 타입

PreToolUse

도구 호출 직전

입력값 검증 · 승인/차단

PostToolUse

도구 실행 직후

결과 검증 · 린트·포매팅

Notification

응답 대기 시

알림 전송 · 외부 연동

Stop

에이전트 턴 종료

최종 정리 · 보고서 생성



Hook 실행 중 Claude는 멈춤. timeout 설정·무거운 작업은 백그라운드(&)로



멀티 인스턴스 & Git Worktree

탭 여러 개 + 워크트리로 진짜 병렬 개발 — 한 저장소에 여러 작업 디렉토리

터미널 탭별 운영

● ● ● 탭 1

Feature-Auth

인증 기능 개발

● ● ● 탭 2

Bug-Fix

긴급 버그 수정

● ● ● 탭 3

Refactor

구조 리팩토링



claude --worktree (네이티브)

```
# 수동 worktree  
git worktree add ../feature feat
```

```
# Claude Code 네이티브 ★  
claude -w feature-auth
```



-w 한 줄로 워크트리 생성 → 브랜치 체크아웃 → 세션 시작까지 자동.
변경 없으면 세션 종료 시 자동 정리.

isolation: worktree → Sub-Agent에서도 격리 가능



💡 생산성 극대화 — 작은 팁 3가지



/voice — 음성 입력

Push-to-talk(스페이스바), 한국어 포함
20개 이상 언어 지원.
키보드 5분짜리 프롬프트를 1분에 전달.



MCP 대신 로컬 Bash

간단한 작업이라면 MCP보다 로컬 스크립트가
훨씬 가볍다. 컨텍스트 공간도 절약.
예) `./scripts/check-db.sh` 실행해



커스텀 MCP 서버

Claude가 작업 중 자동 호출해야 하는
도구가 필요하면 직접 만들기.
예) Plan Review MCP (Gemini로 검토)





통합 데모 — 1차 피드백 한큐에 수정

클라이언트 1차 피드백 10개 항목 → 8단계 파이프라인으로 처리. 수동 만나질 → 30분

1

STAGE

음성 입력

복잡한 피드백 10개를 30초에 전달

2

STAGE

커스텀 MCP (review_plan)

경쟁 모델로 플랜 품질 검증

3

STAGE

Sub-Agent 병렬 실행

UI/테스트 동시 작업

4

STAGE

번들 스킴 (/batch)

P1 4개 파일 병렬 수정

5

STAGE

로컬 Bash 스크립트

test + swiftlint + swiftformat 원커맨드

6

STAGE

Hooks

작업 완료 자동 알림

7

STAGE

코드 리뷰 에이전트

전체 변경사항 품질 검증

8

STAGE

커스텀 스킴 (client-report)

클라이언트 보고서 자동 생성



만나질 작업이 30분으로 — Skills + Sub-Agent + Hooks의 합작



Mash-Up
iOS Team

2026.04.14



Thanks for watching 🙌



<https://www.youtube.com/watch?v=vxEvo2BLM6A>